4/ Déploiement et restitution du projet

1. Déploiement de l'application sur un serveur ou une plateforme cloud

a. Choix de la plateforme de déploiement :

Pour le déploiement de l'application **FashionStore**, on peut utiliser des plateformes cloud populaires comme **Heroku**, **AWS (Amazon Web Services)**, **DigitalOcean**, ou **Vercel**.

b. Préparation de l'application pour le déploiement :

- **Front-end (React) :**
  - Créer une version de production avec la commande npm run build dans le projet React.
  - Cela génère une version optimisée de l'application, prête à être déployée.
- **Back-end (Laravel) :**
  - Vérifiez que votre projet Laravel est prêt pour la production (réglages dans .env, gestion des clés API, etc.).
  - Configurez le serveur de base de données, par exemple, **Heroku Postgres** ou **ClearDB** pour la base de données MySQL.

- **ClearDB** est un service de base de données MySQL dans le cloud. Il permet de gérer des bases de données relationnelles avec MySQL sans avoir à se soucier de l'infrastructure sous-jacente. Ce service est particulièrement utilisé avec des plateformes comme **Heroku**, mais peut aussi être utilisé de manière indépendante.

- **Heroku** est une plateforme cloud **Platform-as-a-Service (PaaS)** qui facilite le déploiement, la gestion et la mise à l'échelle des applications web. Elle prend en charge de nombreuses technologies de développement comme **Node.js**, **Ruby**, **Java**, **PHP**, **Python**, etc.

- **PostgreSQL** (souvent abrégé en **Postgres**) est un **système de gestion de base de données relationnelle** (SGBDR) open-source et puissant. Il permet de stocker et de

gérer des données sous forme de tables, tout en offrant une grande flexibilité et de nombreuses fonctionnalités avancées pour les applications qui nécessitent une gestion de données robuste.

- Optimisez votre application Laravel pour la production en exécutant :

```
php artisan optimize
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

c. Déploiement sur Heroku :

- **Créer un compte sur Heroku** et installer l'outil Heroku CLI sur votre machine locale.

Pour déployer le projet :

1. Initialisez un dépôt Git (si ce n'est pas déjà fait) :

```
git init
```

2. Ajoutez tous les fichiers :

```
git add .
git commit -m "Initial commit"
```

3. Créez une application Heroku :

```
heroku create fashionstore-app
```

4. Déployez l'application sur Heroku :

```
git push heroku master
```

5. Configurez la base de données sur Heroku :

```
heroku addons:create heroku-postgresql:hobby-dev
```

6. Exécutez les migrations Laravel sur Heroku pour configurer la base de données :

heroku run php artisan migrate

- **Déployer le front-end React :**
  - Vous pouvez également déployer le front-end sur **Netlify** ou **Vercel**, deux services gratuits et simples pour déployer des applications React.
  - Vous connectez simplement votre dépôt Git (GitHub, GitLab, Bitbucket) à la plateforme choisie, et elle déploiera automatiquement l'application.

    GitHub et **GitLab** sont des **plateformes de gestion de code source** basées sur **Git**, un système de gestion de versions décentralisé. Ces plateformes permettent aux développeurs de collaborer, partager, et gérer le code source de leurs projets

    Netlify est une plateforme d'hébergement et de déploiement **de sites web statiques** et **d'applications web front-end**. Elle permet de déployer facilement des projets créés avec des frameworks modernes comme React, Vue.js, Angular

    Vercel est une plateforme d'hébergement cloud optimisée pour les **applications front-end** modernes et les **sites web statiques**. Elle est particulièrement bien adaptée aux projets utilisant des frameworks JavaScript comme **Next.js**, qu'elle développe également.

2. Rédiger un guide d'installation et d'utilisation :

Voici un exemple de guide d'installation et d'utilisation pour l'application **FashionStore** :

Guide d'Installation et d'Utilisation de FashionStore

1. **Prérequis :**
   - PHP (>=7.4) et Composer
   - Node.js et npm

- Serveur web (Apache, Nginx) ou Heroku pour le déploiement

2. **Cloner le projet depuis GitHub :**

```
git clone https://github.com/votre-utilisateur/fashionstore.git
cd fashionstore
```

3. **Installation des dépendances Back-end (Laravel) :**
   - Installez les dépendances avec Composer :

```
composer install
```

   - Configurez le fichier .env pour la base de données et autres variables d'environnement.
   - Exécutez les migrations pour configurer la base de données :

```
php artisan migrate
```

4. **Installation des dépendances Front-end (React) :**
   - Allez dans le répertoire du front-end et installez les dépendances :

```
cd front-end
npm install
```

5. **Démarrer l'application en local :**
   - Démarrer le back-end Laravel :

```
php artisan serve
```

   - Démarrer le front-end React :

```
npm start
```

6. **Accéder à l'application en locale :**
   - Accédez à http://localhost:8000 pour le back-end Laravel.

    - Accédez à http://localhost:3000 pour le front-end React.

3. Présentation de l'application lors de la soutenance :

a. Objectifs du projet :

Le projet **FashionStore** a pour objectif de créer une **boutique en ligne** fonctionnelle avec un **front-end dynamique** et un **back-end sécurisé**. Ce projet permet à l'utilisateur de consulter des produits, ajouter des articles au panier, et finaliser des commandes.

b. Choix techniques :

- **Technologies Front-end :**
    - **React** pour sa flexibilité et sa performance dans la création d'interfaces dynamiques.
    - **Bootstrap** pour la mise en page responsive et rapide.
- **Technologies Back-end :**
    - **Laravel** : Framework PHP populaire pour la gestion de l'authentification, des API, et des bases de données avec une structure MVC.
    - **MySQL** ou **Heroku Postgres** : Base de données relationnelle pour stocker les produits, les utilisateurs et les commandes.
- **Sécurisation :**
    - Authentification avec **JWT** pour gérer les sessions et sécuriser les routes sensibles.
    - **HTTPS** pour sécuriser les échanges entre le front-end et le back-end.
    - **Validation des entrées** dans les formulaires pour éviter les attaques par injection SQL.

c. Fonctionnalités clés :

- **Inscription et Connexion** : Les utilisateurs peuvent créer un compte et se connecter pour gérer leurs commandes.
- **Consultation des produits** : Affichage des produits avec options de filtrage.
- **Gestion du panier** : Ajout et suppression de produits au panier.
- **Commande et paiement** : Finalisation de la commande avec un processus de paiement sécurisé.

d. Démonstration en direct :

1. Montrer la page d'accueil avec les produits en vedette.
2. Ajouter un produit au panier et le visualiser.
3. Se connecter, passer une commande et expliquer le traitement des paiements.

e. Challenges techniques :

- Intégration de l’authentification et gestion des rôles utilisateurs.
- Gestion de la sécurité des échanges entre le front-end et le back-end via **JWT**.
- Déploiement de l’application sur **Heroku** avec une base de données cloud.